

En programmation procédurale comme en algorithmique (qui respecte les contraintes fondamentales de la programmation!), **l'ordre des instructions est primordial**. Le processeur exécute les instructions dans l'ordre dans lequel elles apparaissent dans le programme. On dit que l'exécution est **séquentielle**. Une fois que le programme a fini une instruction, il passe à la suivante. Tant qu'une instruction n'est pas terminée, il attend avant de continuer. Par exemple, une instruction de saisie va attendre que l'utilisateur rentre une valeur au clavier avant de continuer.

Parfois, il est nécessaire que le processeur n'exécute pas toutes les instructions, ou encore qu'il recommence plusieurs fois les mêmes instructions. Pour cela, il faudra casser la séquence. C'est le rôle des structures de contrôle. Il existe deux grands types de structures de contrôle :

- ✓ Les *structures conditionnelles (alternatives) ou de choix* vont permettre de n'exécuter certaines instructions que sous certaines conditions
- ✓ Les *structures répétitives ou itératives*, encore appelées boucles, vont permettre de répéter des instructions un certain nombre de fois, sous certaines conditions

1. Les Structures Alternatives (Conditionnelles)

Les structures conditionnelles permettent d'exécuter des instructions différentes en fonction de certaines conditions. Une condition (encore appelée expression conditionnelle ou logique) est évaluée, c'est à dire qu'elle est jugée vraie ou fausse. Si elle est vraie, un traitement (une ou plusieurs instructions) est réalisé ; si la condition est fausse, une autre instruction va être exécutée, et ensuite le programme continuera normalement. En algorithmique, on traduit cette structure alternative à l'aide d'**INSTRUCTIONS CONDITIONNELLES**. Il existe 4 types principaux de structures conditionnelles :

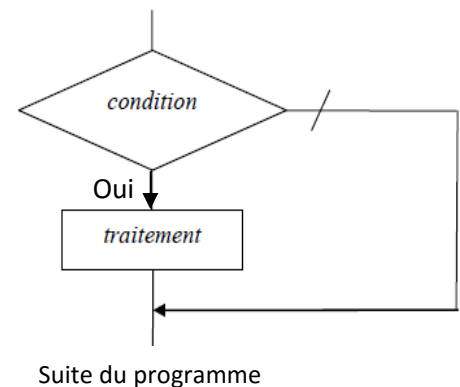
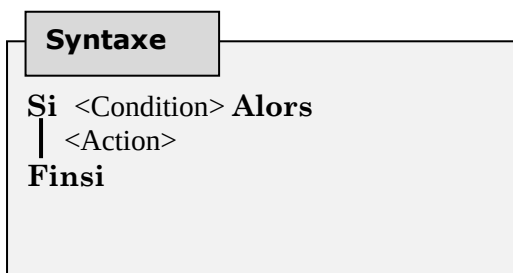
- ✓ Les *structures alternatives simples, unaire ou réduite* (Si...Alors),
- ✓ Les *structures alternatives complètes ou Binaire* (Si...Alors...Sinon),
- ✓ Les *structures alternatives imbriquées*.
- ✓ Les *structures de choix* (Cas où... Vaut) ou (Selon ...).

1.1. L'Alternative Simple Si <condition> alors (if then)

Dans cette structure les actions s'exécutent de la manière suivante :

- ✓ Si la condition <Condition> est vérifiée, l'action (ou le groupe d'actions) <Action> est exécutée puis les actions qui suivent le finSi (la suite du programme).
- ✓ Par contre si la condition n'est pas vérifiée, seules les actions qui suivent le finSi seront exécutées (la suite du programme).

La syntaxe et l'organigramme associé pour une telle structure sont les suivants :



Exemple :

Dans un programme de calcul d'une facture, on veut effectuer une remise de 10% si le montant de la facture dépasse 1000€.

Solution :

Le programme effectue la réduction seulement si le montant est supérieur à 1000€. Sinon, il ne fait aucun traitement particulier et passe à l'instruction suivante. Dans tous les cas, le montant est affiché.

Algorithme Remise ;

VAR

Mont : réel ;

DEBUT

Afficher (" Introduire le Montant : ") ;

Lire (Mont) ;

Si (Mont > 1000) **alors**

 Mont → Mont * 0.9 ;

FinSi ;

Afficher(Mont) ;

Fin.

Program Remise ;

VAR

Mont : real ;

Begin

writeln (' Introduire le Montant : ') ;

Read (Mont) ;

If (Mont > 1000) **then**

 Mont := Mont * 0.9 ;

Write(Mont) ;

End.

Remise.c

#include<stdio.h>

main ()

{

 float Mont ;

printf(" introduire le Montant :\n ") ;

scanf ("%f",&Mont) ;

if Mont > 1000

 Mont = Mont * 0.9 ;

printf("'%f '", Mont) ;

}

1.2. L'Alternative Complète Si <condition> alors sinon (if then else)

Dans cette structure les actions s'exécutent de la manière suivante :

- ✓ Si la condition définie dans <Condition> est vérifiée il faut exécuter l'action (ou le groupe d'actions) définie dans <Action₁>, puis les actions qui suivent le finSi (la suite du programme).
- ✓ Si cette condition n'est pas vérifiée il faut exécuter l'action (ou le groupe d'actions) définie dans <Action₂>, puis les actions qui suivent le finSi (la suite du programme).

La syntaxe et l'organigramme associé pour une telle structure sont les suivants:

Syntaxe

Si <Condition>

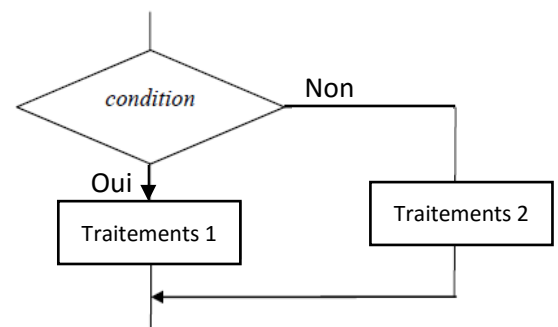
Alors

 <Action₁> (Ins à effectuer si la condition est vérifiée)

Sinon

 <Action₂> (Ins à effectuer si la condition n'est pas vérifiée)

FinSi



Suite du programme

Exemple : Ecrire un algorithme (et en langage Pascal et langage C) qui calcule la valeur absolue d'un nombre ;

Algorithme Valeur_absolue ;

VAR

X: reel ;

DEBUT

Afficher ("Introduire un Réel : ") ;

Lire (X) ;

Si (X ≥ 0)

Alors

Afficher (" La valeur absolue de de ", X, "est : ", X)

Sinon

Afficher (" La valeur absolue de de ", X, "est : ", -X) ;

FinSi

Fin.

```
Program Valeur_absolue ;
VAR
  X: real ;
Begin
  Writeln ("Introduire un Réel : ") ;
  Read (X) ;

  if (X >= 0)
  then
    Write (' la valeur absolue de de', X, 'est : ', X)
  else
    Write (' la valeur absolue de de ', X, 'est : ', -X) ;
End.
```

Valeur_absolue.c

```
#include<stdio.h>
main ()
{
  Float C ;

  printf(" Introduire un réel :\n ") ;
  scanf ("%f",&C) ;

  if (X >= 0)
    printf (" la valeur absolue de de", X, "est : %f ", X) ;
  else
    printf (' la valeur absolue de de ', X, "est : %f ", -X) ;
}
```

1.3. Structures Alternatives Imbriquées

Plusieurs structures alternatives peuvent être imbriquées, si bien que dans un traitement peut figurer une ou plusieurs structure(s) alternative(s). Pour une meilleure lisibilité de l'algorithme, on utilise l'**INDENTATION**, qui consiste à écrire les instructions sur des lignes différentes en procédant à des décalages.

Syntaxe :

```
Si condition1 alors
  Si condition2 alors
    Instructions A
  Sinon
    Instructions B ;
Finsi ;
Sinon
  Si condition3 alors
    Instructions C
  Finsi ;
Finsi
```

Exemple 1 :

Algorithme imbrique ;

```
VAR n : entier ;
DEBUT
  Ecrire ("entrez un nombre : ") ;
  Lire (n) ;
  Si (n < 0) alors
    Ecrire ("Ce nombre est négatif") ;
  Sinon
    Si (n = 0) alors Ecrire ("Ce nombre est nul")
    | Sinon Ecrire ("Ce nombre est positif") ;
  Finsi
Finsi
FIN.
```

Exemple 2 :

Algorithme non_imbrique ;

```
VAR n : entier ;
DEBUT
  Ecrire ("entrez un nombre : ") ;
  Lire (n) ;
  Si n < 0 alors
    Ecrire ("Ce nombre est négatif") ;
  FinSi
  Si n = 0 alors
    Ecrire ("Ce nombre est nul") ;
  FinSi
  Si (n > 0) alors
    Ecrire ("Ce nombre est positif") ;
  Finsi
FIN
```

- **Constat :** Dans l'exemple 2 on fait trois tests systématiquement alors que dans l'exemple 1, si le nombre est négatif on ne fait qu'un seul test
- **Conseil :** utiliser les tests imbriqués pour limiter le nombre de tests et placer d'abord les conditions les plus probables

Remarque :

- La condition ne peut être que vraie ou fausse.
- Dans un programme on met toujours la condition entre parenthèse.
- La condition peut être une expression booléenne simple ou une suite composée d'expressions booléennes (*Une condition composée est une condition formée de plusieurs conditions simples reliées par des opérateurs logiques : ET, OU, OU exclusif (XOR) et NON.*)
- Dans un programme on ne met pas de point-virgule avant sinon.
- Si on a plusieurs instructions à exécuter après "**alors**" ou "**sinon**", il faut toujours les délimiter (borner) avec les mots clés DEBUT et FIN.
- Le **sinon (else)** se rapport toujours au dernier **alors (then)** rencontré auquel un **sinon (else)** n'a pas encore été attribué.

1.4. L'instruction CAS où... VAUT (Case ... of)

On l'utilise lorsque l'on doit comparer une **même** variable avec **plusieurs valeurs**, comme par exemple :

```
Algorithme Exemple ;
VAR
  A: entier ;

DEBUT
  Afficher ("Introduire un entier : ") ;
  Lire (A) ;

  Si (A = 1) Alors
    Instruction1
  Sinon Si (A=2) Alors
    Instruction 2
  Sinon Si (A=3) Alors
    Instruction 3
  Sinon Si (A=4) Alors
    Instruction 4
  Sinon ....
FinSi ;
Fin.
```

Alors, on peut remplacer cette suite de si par l'instruction **CAS où**. Cette instruction permet de sélectionner un groupe précis d'instructions parmi plusieurs, et aussi de réduire la taille d'un code (programme) qui utilise beaucoup d'instruction de **si-alors** pour rendre la logique du programme beaucoup plus lisible.

Syntaxe

```
CAS où <V> VAUT // *case of
  V1 : <action 1> ;
  V2 : <action 2> ;
  V3 : <action 3> ;
  ....
  Vn : <action n> ;
  Autre : <autre action> ;
Fincas
```

- ✓ $V_1, \dots, V_n$ sont des **constantes** de type **scalaire** ou des **expressions ordinales** (entier, naturel, énuméré : intervalle, liste), ou caractère)
- ✓ Action i est exécutée si $V = V_i$ (on quitte ensuite l'instruction cas)
- ✓ Action *autre* est exécutée si quel que soit i , $V \neq V_i$.

L'instruction case diffère de l'instruction Si sur au moins 2 points :

- Le choix peut se faire, non plus seulement entre une ou deux possibilités mais entre un nombre quelconque de possibilités.
- La sélection d'une des possibilités se fait, non plus suivant la valeur d'une expression booléenne, mais suivant la manière dont la valeur d'une expression se situe par rapport à des valeurs (1, 2) ou des intervalles de valeurs (1..10) donnés.

Remarque

- Le nombre de cas, i.e. le nombre de choix possibles d'une instruction case peut très bien se faire (réduire) à un. D'autres part, chaque cas comporte une seule instruction (au sens large) mais il peut s'agir d'un bloc....
- Le type de l'expression doit être *ordinal*, ce qui exclut donc le type réel. Cette restriction est en fait bien compréhensible dans la mesure où des valeurs réelles sont généralement représentées d'une manière approchée ; leur emploi risquerait de conduire à des domaines quelques put ambigus.
- Notez bien que cette instruction *case* (avec ou sans autres (*else*)) commence par le mot réservé *case* et se termine par le mot réservé *end*. Nous rencontrerons d'autres cas où end est associé à autre chose qu'un *Begin*.
- Lorsque la partie *autres* (*else* en pascal) est présente, elle est formée d'une suite d'instructions et non d'un bloc (suite d'instructions borné par début et fin). Bien sûr, il est toujours possible d'en utiliser un, sans que cela ne soit une erreur (puisque un bloc peut toujours apparaître l'où n'importe quelle instruction est autorisée). Simplement, dans ce cas, il faudra prendre garde à ne pas confondre le *end* de ce bloc avec le *end* de *case*.

Exemple :

```
Algorithme Caractere ;
Var C : caractere;
Début
  Afficher (" Tapez un caractère") ;
  Lire (C) ;
  CAS où C VAUT
    'a'..'z' : Afficher ("la lettre est minuscule") ;
    'A'..'Z' : Afficher ("la lettre est majuscule") ;
    Autres : Afficher ("caractère non répertorié") ;
  Fincas ;
Fin.
```

```
Program Caractere ;
Var C : Char;
Begin
  writeln ("Tapez un caractère") ;
  Read ( C ) ;
  Case C of
    'a'..'z' : Write ('La lettre est minuscule') ;
    'A'..'Z' : Write ('La lettre est majuscule') ;
    else : Write ('Caractère non répertorié') ;
  End ;
End.
```

```
Caractere.c ;
#include <stdio.h>
main()
{
  Char C ;
  printf("tapez un caractère \n") ;
  scanf ( "%c ", &C ) ;
  switch C
  {
    'a'..'z' : printf (" La lettre est minuscule") ; break ;
    'A'..'Z' : printf (" La lettre est majuscule") ; break ;
    default : printf (" Caractère non répertorié") ;
  }
}
```

1.4.1. L'ordre Des Tests Dans L'instruction Cas (Case...of)

D'une manière générale, l'instruction case explore les différents domaines dans l'ordre où ils sont mentionnés. Dès qu'elle en a trouvé un qui convient, elle exécute l'instruction correspondante et ne cherche pas à poursuivre son exploration. Supposons qu'on a l'instruction suivante :

```
Case n of
  1..5 : ..... {1}
  3..10 : ..... {2}
End ;
```

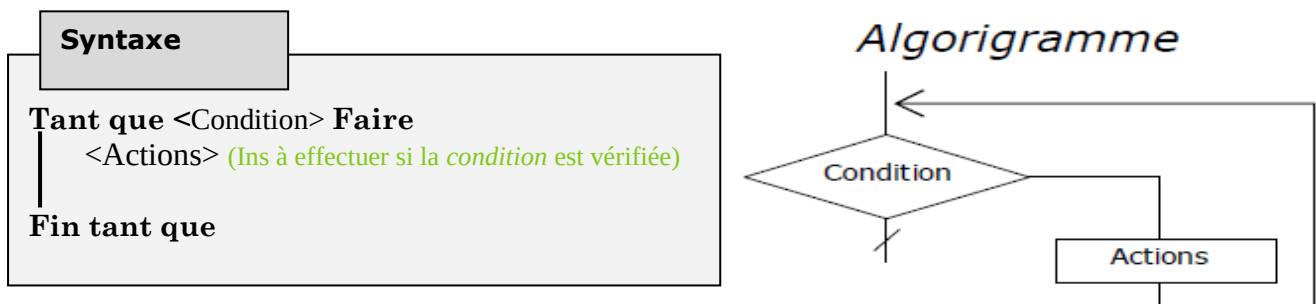
On constate que les deux domaines 1..5 et 3..10 possèdent les valeurs 3,4 et 5 en commun. Que se passe-t'il lorsque n vaut 4 ? en fait, il y a exécution de l'instruction {1} puis passage à l'instruction suivante. Ce qui signifie qu'ici l'instruction {2} ne pourra jamais être exécutée lorsque n vaut 3, 4 ou 5.

2. Les Structures Répétitives ou Itératives (Les Boucles)

Un programme a presque toujours pour rôle de répéter la même action un certain nombre de fois. Pour ce faire, on utilise une structure permettant de dire « Exécuter telles actions jusqu'à ce que telle condition soit remplie ». Bien qu'une seule soit nécessaire, la plupart des langages de programmation proposent trois types de structures répétitives.

2.1. La Structure Tant que <Condition> Faire (While Do)

Parfois, pour réaliser une tâche, on doit effectuer plusieurs fois les mêmes instructions, sans que le nombre de fois soit déterminé à l'avance. On utilise alors une **BOUCLE CONDITIONNELLE**. Dans cette structure, le même traitement est effectué tant qu'une condition reste valide ; la boucle s'arrête quand celle-ci n'est plus remplie. La syntaxe et l'Algorigramme associé pour une telle structure sont ainsi formulés :



Remarque

- ✓ Le nombre de répétitions dépendra de la *condition*.
- ✓ Si la condition n'est pas vérifiée au début, alors les <actions> ne seront pas exécutées du tout.
- ✓ Si la *condition* est vérifiée au début et si la *condition* n'est pas susceptible d'être modifiée lors des <actions>, alors les <actions> seront indéfiniment exécutées et l'utilisateur sera contraint d'arrêter le programme. Dans ce cas, il s'agit d'une erreur majeure car un programme ne doit pas boucler indéfiniment mais au contraire s'arrêter automatiquement une fois que la *condition* cesse d'être vérifiée.

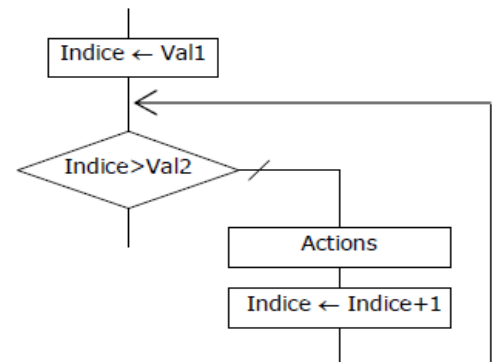
2.2. La Structure Pour <Indice> allant de ... à ... Faire (for i := 1 to n do)

Cette structure est une **BOUCLE ITERATIVE**, elle consiste à répéter une action (ou groupe d'actions) un nombre de fois déterminé par les valeurs initiale et finale de paramètre <indice>. La partie action est exécutée la première fois en affectant au paramètre <indice> que l'on appelle aussi compteur, une valeur <val_initiale>. Après chaque exécution on rajoute au compteur la valeur 1. L'action est ré-exécutée jusqu'à ce que la valeur du compteur dépasse une valeur spécifiée par <val_finale>. La syntaxe et l'Algorithme associé pour une telle structure sont ainsi formulés :

Syntaxe

```
Pour <Indice> allant de <val_initiale> à <val_finale> Faire  
|  
| <Actions>  
Fin Pour
```

Algorithme



Remarque :

- ✓ La variable < indice > est un **compteur**, dont la valeur augmente automatiquement de 1 à chaque tour. Cette variable permet en définitive de contrôler le nombre entier de tours. Cette variable est en d'autres termes une variable de contrôle d'itération, caractérisée par sa valeur initiale, sa valeur finale et son pas de variation.
- ✓ La **sortie de la boucle** s'effectue lorsque le nombre souhaité d'itérations est atteint, c'est-à-dire lorsque prend la valeur N.
- ✓ Les valeurs initiale (Val₁) et finale (Val₂) sont incluses dans le comptage. Il est éventuellement possible de spécifier un autre pas d'incréméntation (+2,+10,-1....) : **POUR** <indice> **Allant de** Val₁ **A** Val₂ [**PAS** Valeur du pas] **Faire**.

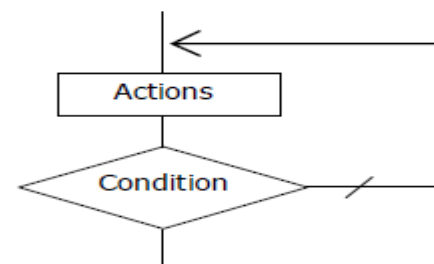
2.3. La Structure Répéter ... Jusqu'à <Condition> (Repeat until)

Cette Structure permet de répéter une action (groupe d'actions) jusqu'à ce que la condition soit vérifiée. La syntaxe et l'Algorithme associé pour une telle structure sont ainsi formulés :

Syntaxe

```
Répéter  
| <Actions>  
Jusqu'à <condition>
```

Algorithme



Remarque

- ✓ La vérification de la condition s'effectue **après** les actions. Celles-ci sont donc exécutées au moins une fois à l'inverse de la boucle répétitive **Tant que** ou les actions ne peuvent être exécutées.
- ✓ Dans <actions> au moins une instruction doit porter sur les données servant à la modification de la condition.
- ✓ Pas besoin d'encadrer une suite d'instructions par un **Begin** et **End**, le **repeat-until** joue déjà ce rôle.

Exemple : Calculer la somme des N premiers nombres.

Algorithme Exemple1 ;

VAR

N, S, I: entier ;

DEBUT

Afficher ("Introduire un Entier : ") ;

Lire (N) ;

$I \rightarrow 1$; $S \rightarrow 0$;

Tant que ($I \leq N$) **Faire**

DEBUT

$S \rightarrow S + I$;

$I \rightarrow I + 1$;

FIN ;

Afficher ("La Somme des ", N, "premier(s) nombres est : ",

FIN.

Algorithme Exemple2 ;

VAR

N, S, I: entier ;

DEBUT

Afficher ("Introduire un Entier : ") ;

Lire (N) ;

$S \rightarrow 0$;

Pour I allant de 1 à N Faire

$S \rightarrow S + I$;

Afficher ("La Somme des ", N, "premier(s) nombres est : ", S) ;

FIN.

2.4. Choix d'un type de Boucle

- Si on peut déterminer le nombre d'itérations avant l'exécution de la boucle, il est plus naturel d'utiliser *la boucle Pour*
- S'il n'est pas possible de connaître le nombre d'itérations avant l'exécution de la boucle, on fera appel à l'une des *boucles TantQue ou répéter jusqu'à*
- Pour le choix entre *TantQue* et *jusqu'à* :
 - ✓ Si on doit tester la condition de contrôle avant de commencer les instructions de la boucle, on utilisera *TantQue*
 - ✓ Si la valeur de la condition de contrôle dépend d'une première exécution des instructions de la boucle, on utilisera *répéter jusqu'à*